AutoGrader: A Rule-Driven Evaluator for C Programs

LAB PROGRESS REPORT (agentic01.pdf) • COMPUTING LABORATORY

---

1. Executive Summary & Reproducibility Contract

Manual grading of student C programming submissions is slow, subjective, and difficult to scale across large class sizes. **AutoGrader** is an autonomous, rule-driven evaluation agent designed to judge C programs unattended under fixed time and memory limits. It reads grading rules from an external specification file, performs a cheap first-pass sanity sweep, dispatches nine specialized expert analyzers in parallel, and returns an explainable grade with actionable feedback.

The agent strictly honors the **Reproducibility Contract**: *The same submission judged under the same rules receives the identical grade every time the agent is executed*.

2. Group Workload Distribution

Development responsibilities are partitioned cleanly across four team members according to architectural tiers:

Member	Subsystem & Role	Deliverables & Implementation Scope
Member 1 (Lead)	Core Architecture & Process Sandboxing	• Main Coordinator (coordinator.py) • Process supervisor & timeouts (sandbox.py) • First-pass sanity sweep (compile, smoke, static) • Command Line Interface (cli.py)
Member 2	Execution, Safety & Counter-Tests	• Hidden I/O test runner (correctness.py) • Boundary & overflow runner (logical.py) • Memory safety & ASan/UBSan (memory_safety.py) • Counter-test generator (counter_test_generator.py)
Member 3	Static Analysis & Complexity	• Compiler warnings engine (syntax.py) • Cyclomatic complexity & length (structure.py) • Modularity & helper split (modularity.py) • Complexity timing curve (complexity.py)
Member 4	Originality, LLM Style & Reporting	• Token Winnowing similarity (originality.py) • Deterministic LLM style review (readability.py) • JSON / TXT report collation (reporter.py) • Unit test suite & PDF generator (agentic01.pdf)

3. Implementation Progress Achieved So Far

A. First-Pass Cheap Sanity Sweep: Verifies static structure (main, readers, writers, return), runs compiler checks with -O2 -Wall -Wextra -std=c11, and executes a 1-second smoke run. If an infinite loop is detected, the agent sets suspected_infinite_loop: true, skips dynamic execution, and outputs a partial report.
B. Nine Expert Specialist Analyzers: Fully operational in autograder/analyzers/ (correctness, syntax, structure,

modularity, logical, complexity, memory safety, readability, originality).
C. Counter-Test Synthesis & Deduction Analysis: Synthesizes concrete failing inputs and detailed on-point deductions explaining why points decreased.
D. Multi-Solution Special Judge: Evaluates problems with multiple valid answers (e.g. Two-Sum) by mathematical verification instead of rigid diffing.
E. Problem-Specific Report Generation: Tailors usefulness and limitations dynamically per problem domain.
F. Minimalist Web UI (Dark/Light Mode): Flask web portal on local host with live execution, deduction breakdowns, and report downloads.
G. Full Automated Unit Test Suite: 9 comprehensive tests running with 100% pass rate in unit_tests/run_all_tests.py.

4. Benchmark Results on Multi-Domain Problem Suites

Submission File	Rule Set Target	Grade	Diagnostic Verification
add_student.c	add_two_integers	**79.0 / 100**	Main function only; fails signed overflow on INT_MAX+1; terse variables.
add_perfect.c	add_two_integers	**100.0 / 100**	Modular helper function; clean headers; 64-bit integer overflow protection.
add_infinite_loop.c	add_two_integers	**38.0 / 100**	Smoke run timed out; suspected_infinite_loop set; dynamic tests bypassed.
add_plagiarized.c	add_two_integers	**69.0 / 100**	Similarity 1.00 against add_student.c; flagged above 0.85 threshold.
two_sum_quadratic.c	two_sum_sorted	**66.0 / 100**	Nested loops flagged: strictly violates O(N) complexity bound requirement.
two_sum_optimal.c	two_sum_sorted	**95.0 / 100**	Optimal O(N) two-pointer technique in helper; Special Judge validated multi-solution.
stack_leaky.c	dynamic_stack	**78.7 / 100**	Memory safety penalized: allocation without matching free(); monolithic main.
stack_robust.c	dynamic_stack	**96.0 / 100**	6 helper functions; underflow protection; zero memory leaks confirmed.
palindrome_buggy.c	palindrome_checker	**85.5 / 100**	Fails case-insensitivity on mixed casing; flagged by synthesized counter-test.
palindrome_perfect.c	palindrome_checker	**90.0 / 100**	Clean helper function; alphanumeric normalization; passes all edge tests.
binary_search_linear.c	binary_search	**83.7 / 100**	O(N) linear scan penalized; fails logarithmic complexity bound requirement.
binary_search_optimal.c	binary_search	**96.0 / 100**	Iterative O(log N) bisection helper; clean contracts and edge checks.

5. Usefulness & Limitations in Real-Life Environments

Real-Life Usefulness:

1. *High-Volume Technical Hiring*: 500 candidate submissions evaluated in under 15 minutes with explainable per-metric justification.
2. *Plagiarism & LLM Triage*: Computes token n-gram similarity against 40,000+ pairs, flagging only anomalous pairs (>0.85).
3. *24/7 Formative Lab Feedback*: Provides instant counter-test feedback to students at 2 AM without waiting days for TA grading.
4. *CI/CD Regression Gating*: Pre-commit gate for embedded software ensuring 100/100 across sanitizers and complexity bounds.

Fundamental Limitations & Concrete Mitigations:

1. *Tests cannot prove correctness*: A submission passing 10 hidden tests may fail on untested whitespace or trailing newlines. **Mitigation**: Incorporate automated fuzzing (libFuzzer) and reference-model counter-test generation.
2. *Timing curves sensitive to OS noise*: Background OS indexing can distort wall-clock empirical curves from O(1) to O(N). **Mitigation**: Profile hardware CPU instruction cycles and normalize against an in-slice baseline kernel.
3. *Token similarity evadable by macro obfuscation*: Reordering statements or wrapping expressions in macros reduces lexical n-gram similarity. **Mitigation**: Compare Abstract Syntax Tree (AST) tree-isomorphisms and LLVM bitcode diffs.
4. *Compiler optimization masking bugs*: GCC -O2 can optimize away uninitialized variables or dead loops. **Mitigation**: Run multi-tier compilation (-O0 with ASan/UBSan, -O2 with strict warnings).

6. Project Timeline & Next Milestones

Milestone	Target	Deliverables & Strategic Objectives
Milestone 1 (Complete)	Week 1	Architecture, sandbox, external rules parser, 9 expert analyzers, benchmark reports, agentic01.pdf.
Milestone 2	Week 2	Automated fuzz testing engine, AST graph isomorphism for plagiarism detection, multi-tier compiler matrix.
Milestone 3	Week 3	Hardware performance counter profiling, multi-file project support (Makefiles), local Ollama LLM integration.
Milestone 4 (Final)	Week 4	Interactive instructor dashboard, 15-20 min demonstration video on YouTube, final report submission.